

DS-GA 1004 Capstone Project: Goodreads Recommendation and Market Segmentation

Ishan Malik (im2854)

Yihe Huang (yh5050)

May 15, 2026

1 Link to Group's GitHub Repository & Contributions

GitHub Repository: https://github.com/nyu-big-data/capstone-cap_11.git

Group Contributions

- **Ishan Malik:** Led the data engineering pipeline, Parquet conversion, and ALS hyperparameter tuning. Formulated the combined explicit/implicit feedback scoring function.
- **Yihe Huang:** Implemented MinHash for Market Segmentation and generated similarity metrics. Developed the popularity baseline and evaluation framework (NDCG/MAP).

AI Contributions

Windsurf was utilized as an agentic development tool to prototype PySpark pipelines, refactor cluster submission scripts, and debug HDFS file operations. Gemini and Claude were used to clarify doubts during implementation.

2 List of Top 100 Most Similar Pairs

2.1 Method

To identify books that occupy similar positions in the reading market, we operationalise similarity as the Jaccard similarity between the sets of users who have read each book:

$$J(A, B) = \frac{|U_A \cap U_B|}{|U_A \cup U_B|}$$

where U_A and U_B are the sets of users who read books A and B respectively. A score of 1 indicates identical readership; 0 indicates no overlap.

Computing exact Jaccard similarity over all book pairs at this scale is infeasible. We therefore use **MinHash LSH** (PySpark's `MinHashLSH` with 20 hash tables) to efficiently approximate the similarity join. Each book is represented as a sparse binary vector of dimension N_u (total number of users), with a 1 at each position corresponding to a user who read that book. An approximate similarity join is performed with a Jaccard distance threshold of 0.8 (i.e., similarity ≥ 0.2), and the top 100 pairs by similarity are retained.

2.2 Top 100 Most Similar Book Pairs

Table 1: Top 100 most similar book pairs by Jaccard similarity.

Book A	Book B	Jaccard Distance	Similarity
15537	15538	0.0899	0.9101
32595	32597	0.0909	0.9091
3908	3911	0.1034	0.8966
58993	58994	0.1111	0.8889
42701	42704	0.1111	0.8889
15536	15537	0.1149	0.8851
3910	3911	0.1167	0.8833
42701	42702	0.1207	0.8793
15539	15541	0.1237	0.8763
42702	42704	0.1271	0.8729
13977	13978	0.1284	0.8716
15535	15538	0.1319	0.8681
12807	12809	0.1322	0.8678
15540	15541	0.1327	0.8673
15536	15538	0.1333	0.8667
13974	13975	0.1346	0.8654
13976	13977	0.1376	0.8624
6405	6407	0.1418	0.8582
66	1116	0.1426	0.8574
42704	42705	0.1440	0.8560
13978	13979	0.1441	0.8559
42704	47240	0.1466	0.8534
13975	13977	0.1481	0.8519
12796	12797	0.1505	0.8495
3908	3910	0.1525	0.8475
13319	13320	0.1532	0.8468
15535	15537	0.1556	0.8444
13979	13980	0.1557	0.8443
15533	15534	0.1558	0.8442
15538	15539	0.1563	0.8438
6646	6647	0.1566	0.8434
6182	6184	0.1567	0.8433
13975	13976	0.1574	0.8426
3906	3908	0.1579	0.8421
13316	13318	0.1581	0.8419
27579	59022	0.1600	0.8400
2615	2616	0.1605	0.8395
17318	17319	0.1607	0.8393
13692	13695	0.1639	0.8361
19978	20704	0.1639	0.8361
13699	17444	0.1650	0.8350
6184	6185	0.1655	0.8345
58994	58995	0.1667	0.8333

Book A	Book B	Jaccard Distance	Similarity
6402	6403	0.1667	0.8333
42702	42705	0.1680	0.8320
42701	42705	0.1680	0.8320
5674	5677	0.1692	0.8308
20692	20696	0.1695	0.8305
66	1117	0.1702	0.8298
13976	13978	0.1712	0.8288
15539	15540	0.1717	0.8283
6535	6536	0.1719	0.8281
42703	42705	0.1742	0.8258
19387	20696	0.1774	0.8226
15535	15536	0.1778	0.8222
15537	15539	0.1789	0.8211
58963	59022	0.1789	0.8211
14240	17443	0.1789	0.8211
6252	6254	0.1791	0.8209
13699	14241	0.1802	0.8198
12809	12810	0.1832	0.8168
3906	3911	0.1833	0.8167
5358	5359	0.1839	0.8161
46538	46539	0.1857	0.8143
15099	15540	0.1869	0.8131
42702	47240	0.1880	0.8120
42701	47240	0.1880	0.8120
12797	12800	0.1881	0.8119
15534	15536	0.1882	0.8118
50810	50811	0.1884	0.8116
28069	28071	0.1895	0.8105
13696	13697	0.1905	0.8095
21090	21091	0.1905	0.8095
939	944	0.1911	0.8089
613	944	0.1913	0.8087
13318	13319	0.1913	0.8087
6478	6479	0.1918	0.8082
15538	15541	0.1919	0.8081
15533	15536	0.1928	0.8072
938	941	0.1944	0.8056
7228	7229	0.1948	0.8052
12804	12807	0.1983	0.8017
39208	46574	0.2000	0.8000
3906	3910	0.2000	0.8000
59022	59023	0.2000	0.8000
1386	1387	0.2012	0.7988
42705	47240	0.2016	0.7984
13316	13317	0.2019	0.7981
28068	28069	0.2021	0.7979
15534	15538	0.2022	0.7978

Book A	Book B	Jaccard Distance	Similarity
55	46571	0.2031	0.7969
39208	46573	0.2051	0.7949
13320	13321	0.2061	0.7939
13316	13319	0.2063	0.7937
17442	17443	0.2065	0.7935
1116	1117	0.2066	0.7934
15534	15537	0.2069	0.7931
938	944	0.2070	0.7930
42703	42704	0.2077	0.7923
6399	6400	0.2077	0.7923

The IDs shown are Goodreads internal book identifiers. The prevalence of numerically adjacent IDs (e.g. 15535–15541, 42701–42705, 13974–13980) strongly suggests these are books within the same series or by the same author, whose readership naturally overlaps almost entirely.

3 Documentation of Train/Validation Split Generation

3.1 Dataset Overview

The raw interaction data were sourced from the Goodreads dataset assembled by Wan and McAuley (2018). The full dataset contains approximately 876 000 unique users, 2.4 million unique books, and 223 million user–book interaction records. Each record has the following schema:

`user_id, book_id, is_read, rating, is_reviewed`

The three feedback signals are interpreted as follows:

- **rating** — an integer star rating (1–5); the primary *explicit* preference signal.
- **is_read** — a binary flag indicating that the user marked the book as read; treated as an *implicit consumption* signal.
- **is_reviewed** — a binary flag indicating that the user wrote a review; treated as an *implicit engagement* signal.

3.2 Ingestion and Format Conversion

The raw interactions were distributed as a single CSV file stored in HDFS at `/user/pw44_nyu_edu/goodreads_interactions.csv`. As an initial preprocessing step (`preprocess.py`), we converted this file to Apache Parquet using PySpark and wrote it to our personal HDFS directory:

```
interactions = spark.read.csv(
    "hdfs:///user/pw44_nyu_edu/goodreads_interactions.csv",
    header=True, inferSchema=True
)
interactions.write.mode("overwrite").parquet(
    "hdfs:///user/im2854_nyu_edu/goodreads/interactions.parquet"
)
```

The Parquet representation is used for all downstream computation. Compared with CSV, it reduces I/O through columnar storage and native compression, avoids repeated schema inference at read time, and is partitioned across the HDFS cluster for efficient distributed Spark jobs.

3.3 User Filtering

Because users with very few interactions cannot be meaningfully split into training and evaluation portions, we first filtered out any user whose total interaction count fell below ten:

```
user_counts = df.groupBy("user_id").agg(count("*").alias("cnt"))
valid_users = user_counts.filter(col("cnt") >= 10)
df_filtered = df.join(valid_users.select("user_id"),
                      on="user_id", how="inner")
```

This minimum-activity threshold ensures that every retained user contributes a sufficient number of interactions to each of the three downstream partitions while still leaving a meaningful training signal per user.

3.4 Within-User Interaction Split

Rather than partitioning users into disjoint groups, we adopted a **within-user interaction split**: every retained user contributes interactions to all three splits (train, validation, and test). This design ensures that the model has seen some history for every user that appears in the evaluation sets, making it possible to generate personalized recommendations for each evaluated user.

The split was implemented in `split_data.py` as follows:

1. A row number was assigned to each interaction within each user’s history using a *randomised* ordering (PySpark `rand(seed=42)`, partitioned by `user_id`):

```
window_spec = Window.partitionBy("user_id").orderBy(rand(seed=42))
df_ranked    = df_filtered.withColumn("row_num",
                                      row_number().over(window_spec))
```

2. A fractional rank p was computed for each interaction as $p = \text{row_num}/\text{cnt}$, where `cnt` is the total number of interactions for that user.
3. Interactions were assigned to partitions based on p :

- | | |
|---------------------|--------------------|
| • Train | $p \leq 0.8$ |
| • Validation | $0.8 < p \leq 0.9$ |
| • Test | $p > 0.9$ |

Table 2 summarises these parameters.

Table 2: Train/validation/test split parameters.

Parameter	Value	Notes
Minimum interactions per user	10	Users below threshold discarded
Splitting unit	Per-user interaction	Every user appears in all splits
Interaction ordering	Random (<code>rand</code> , seed = 42)	Reproducible
Train fraction	80% of each user’s interactions	Model fitting
Validation fraction	10% of each user’s interactions	Hyperparameter tuning
Test fraction	10% of each user’s interactions	Final evaluation
Storage format	Apache Parquet (HDFS)	

3.5 Reproducibility and Storage

Using a fixed seed (`seed=42`) for the random ordering makes the partition assignment fully deterministic and reproducible. Each split was materialised as a separate Parquet dataset in HDFS:

Path	Contents
<code>.../goodreads/train.parquet</code>	80% of each user’s interactions
<code>.../goodreads/val.parquet</code>	Middle 10% of each user’s interactions
<code>.../goodreads/test.parquet</code>	Final 10% of each user’s interactions

Pre-materialising the splits ensures that every model (popularity baseline, explicit ALS, implicit ALS, and the combined model) is trained and evaluated on identical data, making all metric comparisons directly meaningful.

3.6 Evaluation Protocol

At evaluation time, the 80% train partition serves as the *observed history* for each user; the model generates a ranked list of top-100 book recommendations from items *not* seen in that user’s training interactions. The held-out validation or test partition provides the set of ground-truth relevant items against which ranking metrics (Precision@K, MAP, NDCG) are computed.

4 Additional Pre-processing

Type casting. Before fitting any model, `user_id` and `book_id` are cast to `int` and `rating` to `float`, as required by Spark’s ALS implementation. `is_read` and `is_reviewed` are cast to `float` where used arithmetically.

Removal of unrated interactions. The explicit ALS model is trained only on rows where `rating IS NOT NULL AND rating > 0`, discarding interactions where a user read a book without leaving a star rating. Retaining null or zero ratings would bias the model toward predicting zero as a valid score.

Relevance thresholds for evaluation. A held-out interaction is counted as relevant only if it represents a clear positive preference. The thresholds used are: `rating ≥ 4` for the explicit model; `is_read = 1` for the implicit model; and the union of both conditions for the combined model.

Training filter (implicit model). The implicit model is trained only on interactions where the user both read *and* reviewed the book (`is_read = 1` **and** `is_reviewed = 1`), focusing on the most engaged users and reducing dataset size for scalability.

Confidence score (implicit model). Following Hu, Koren, and Volinsky (2008), a confidence weight is constructed as:

$$c_{ui} = 1 + 40 \cdot (\text{is_read}_{ui} + \text{is_reviewed}_{ui})$$

Because the training filter guarantees both flags equal 1, the confidence is constant at $1 + 40 \times 2 = 81$ for every training interaction.

Training filter (combined model). The combined model applies a different filter: only interactions where the user both *rated* and *reviewed* the book are retained (`rating > 0` **and** `is_reviewed = 1`). This restricts training to the highest-quality explicit preference signals — interactions backed by both a star rating and written engagement.

Combined signal. After the filter, every retained row has `rating > 0` and `is_reviewed = 1`, so the derived columns simplify as follows. The `effective_rating` is always the actual star rating (the `is_read` and `otherwise` fallback branches never fire). The confidence weight evaluates to $1 + 40 \times (1 + 1) = 81$ for every row, since `is_reviewed` is always 1 and the `rating > 0` branch is always taken.

Popularity score. The popularity baseline ranks books by $\bar{r}_i \cdot \ln(\text{count}_i + 1)$, where $\bar{r}_i$ is the mean rating and `counti` is the number of explicit ratings. The log dampening prevents high-volume but mediocre books from dominating.

Market segmentation preprocessing. For MinHash LSH, only `is_read = 1` interactions are retained. User IDs are mapped to a contiguous integer range via `StringIndexer`, and each book is represented as a sparse binary vector of dimension N_u with a 1 at each index corresponding to a user who read it. Jaccard similarity between two such vectors equals the fraction of shared readers.

Cold-start handling. All ALS models set `coldStartStrategy = "drop"`, removing predictions for users or books absent from training to prevent NaN values from corrupting ranking metrics.

5 Evaluation of Popularity Baseline

5.1 Method

The popularity baseline ranks all books by a score that balances mean rating against interaction volume:

$$\text{pop_score}_i = \bar{r}_i \cdot \ln(\text{count}_i + 1)$$

where $\bar{r}_i$ is the mean explicit rating (1–5 stars) and `counti` is the number of explicit ratings for book i . The logarithmic dampening prevents highly-rated books with very few ratings from outranking broadly popular titles. The top 100 books by this score are identified from the training set and recommended identically to every user. A held-out interaction is treated as relevant if `rating` ≥ 4 .

5.2 Results

Table 3: Popularity baseline performance at cutoff $k = 100$.

Split	MAP@100	NDCG@100
Validation	0.0203	0.0653
Test	0.0207	0.0664

5.3 Discussion

The scores are low across all three metrics, which is expected for a non-personalised recommender: every user receives the same list regardless of their reading history. The method captures only aggregate popularity and cannot account for individual taste. Nevertheless, it serves as a meaningful lower bound; any personalised model that cannot surpass these figures would provide no benefit over a simple global chart. The close agreement between validation and test scores (MAP 0.0203 vs. 0.0207, NDCG 0.0653 vs. 0.0664) confirms that the held-out sets are drawn from the same distribution and that the baseline is stable.

6 Documentation of Latent Factor Models Hyper-parameters and Validation

6.1 Hyperparameters

All three latent-factor models use Spark’s ALS implementation and share the same core hyperparameters: **rank** (dimensionality of latent factors), **regParam** (L2 regularisation), and **maxIter** (number of alternating least-squares iterations). The implicit and combined models additionally use the **confidence scale** α , which controls how strongly behavioural signals inflate the confidence weight relative to the background level of one.

Hyperparameter selection used NDCG@100 on the validation set as the tuning criterion, since NDCG captures both the presence and the rank position of relevant items in the top-100 list. The final configuration for all three models is:

Table 4: Final hyperparameter values for all ALS models.

Model	Rank	regParam	maxIter	α
Explicit ALS	100	0.1	10	—
Implicit ALS	100	0.1	10	40
Combined ALS	100	0.1	5	40

A rank of 100 provides enough capacity to capture diverse reading preferences without overfitting on a dataset of this scale. A regParam of 0.1 is a standard value that prevents latent factors from growing unbounded while still allowing the model to fit the training data closely. $\alpha = 40$ follows the recommendation of Hu, Koren, and Volinsky (2008) as a reasonable default for implicit feedback datasets. The combined model uses **maxIter** = 5 rather than 10 due to cluster time constraints; the other two models use 10 iterations.

7 Evaluation of Latent Factor Models

7.1 Evaluation Protocol

Each model generates a ranked list of the top 100 books for each user, excluding books already seen in that user’s training interactions. Evaluation is performed against the held-out validation and test interactions using three ranking metrics from Spark’s `RankingMetrics` API:

- **MAP@100** — Mean Average Precision, rewarding models that place relevant items early in the ranked list.
- **NDCG@100** — Normalised Discounted Cumulative Gain, which applies a logarithmic discount to lower-ranked relevant items.
- **Precision@100** — fraction of the top-100 recommendations that are relevant.

Ground-truth relevance is defined per model as described in Section 4: `rating` ≥ 4 for the explicit model, `is_read` = 1 for the implicit model, and the union of both for the combined model. Due to cluster memory constraints, evaluation is capped at 10,000 users per run; the `coldStartStrategy` = "drop" setting removes any user or book absent from training before metrics are computed.

7.2 Results

Table 5: Ranking performance of all models at cutoff $k = 100$.

Model	Split	MAP@100	NDCG@100	P@100
Popularity baseline	Validation	0.0203	0.0653	0.0088
	Test	0.0207	0.0664	0.0089
Explicit ALS	Validation	0.0007	0.0020	0.0002
	Test	0.0008	0.0018	0.0002
Implicit ALS	Validation	0.0226	0.0814	0.0177
	Test	0.0230	0.0821	0.0180
Combined ALS	Validation	0.0239	0.0830	0.0176
	Test	0.0233	0.0827	0.0177

7.3 Discussion

The explicit ALS model underperforms the popularity baseline by a substantial margin across all metrics (NDCG@100: 0.0020 vs. 0.0653 on validation). This is a notable result: a non-personalised global chart outperforms a trained latent-factor model. Several factors likely contribute. First, the explicit model is trained only on rated interactions, which represent a small and potentially unrepresentative fraction of all user activity — many reads go unrated. Second, with a 10-iteration ALS run on a sparse signal, the learned latent factors may not have fully converged. Third, the ground-truth threshold of `rating` ≥ 4 means only high ratings count as relevant, making it harder for any model to score well when the relevant set per user is small.

The implicit ALS model substantially outperforms both the explicit ALS model and the popularity baseline (NDCG@100: 0.0814 vs. 0.0653 on validation). Training on interactions where users

both read *and* reviewed a book focuses the model on the most engaged segment of the user population, producing a cleaner and more consistent preference signal than either the sparse explicit ratings or the full unfiltered implicit data would provide. The strong test-set scores (MAP 0.0230, NDCG 0.0821) confirm that this improvement generalises beyond the validation set.

The combined ALS model achieves the best overall performance, narrowly outperforming the implicit model on validation (NDCG@100: 0.0830 vs. 0.0814) and remaining competitive on test (0.0827 vs. 0.0821). This improvement is modest but consistent, suggesting that incorporating explicit star ratings as the **effective_rating** signal — on top of the high-confidence filter of rated and reviewed interactions — adds a small but meaningful quality signal beyond behavioural engagement alone. Notably, this was achieved with only 5 ALS iterations (vs. 10 for the other models) due to cluster time constraints, so further gains may be possible with more iterations.

8 Additional Software Components

No external distributed computing libraries outside of standard **PySpark** (Spark SQL, MLlib) were utilized for model training.